



Politechnika Łódzka
Instytut Automatyki

Laboratorium Robotów Mobilnych

Instrukcja 3

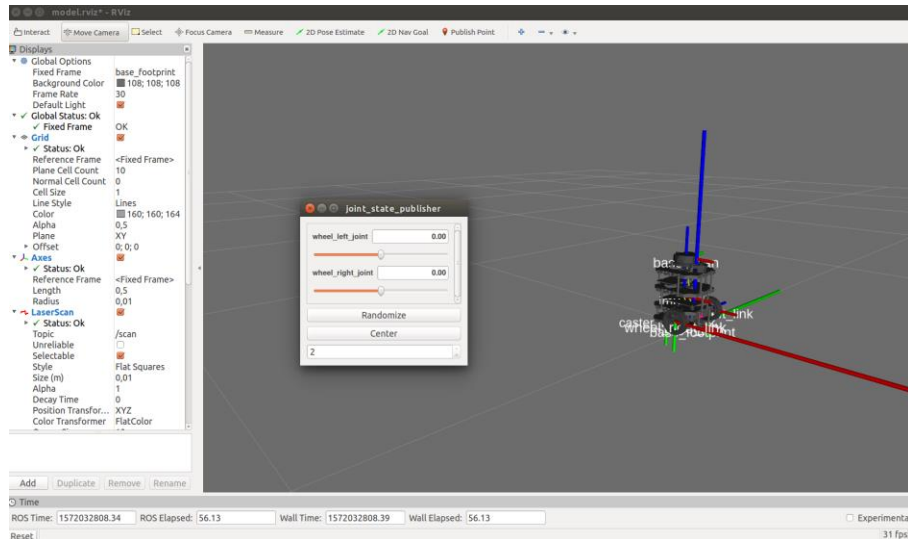
Łódź, 2019

Wstęp

Celem niniejszych zajęć jest zaznajomieniem się z transformatami współrzędnych w ROSie. Do tego służy paczka tf, która pozwala na śledzenie wielu układów współrzędnych w czasie.

Aby zrozumieć ideę układów współrzędnych otwieramy nowy terminal i wpisujemy:

```
roslaunch turtlebot3_bringup turtlebot3_model.launch
```



Na poziomie okna w Global Options wybieramy układ base_footprint.

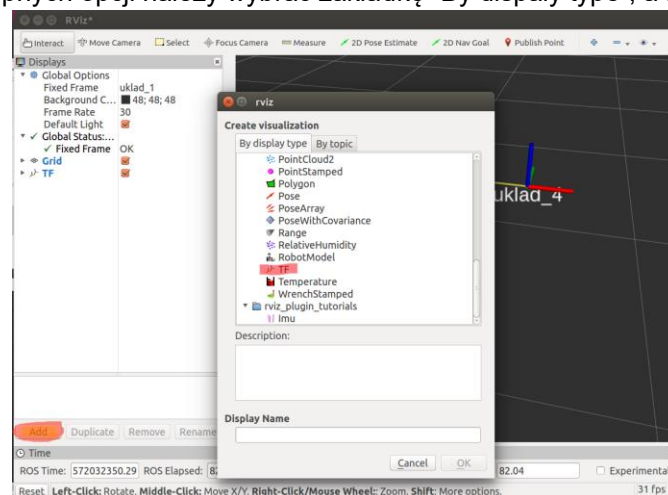
Broadcaster

Przykład 1

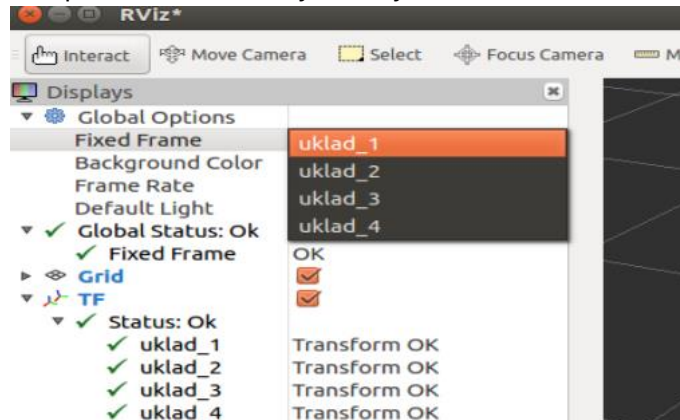
Uruchamiamy pierwszy przykład pod nazwą tf_broadcaster.ipynb. Następnie z poziomu terminalu uruchamiamy RVIZ'a. Poprzedniego można zamknąć.

```
rviz
```

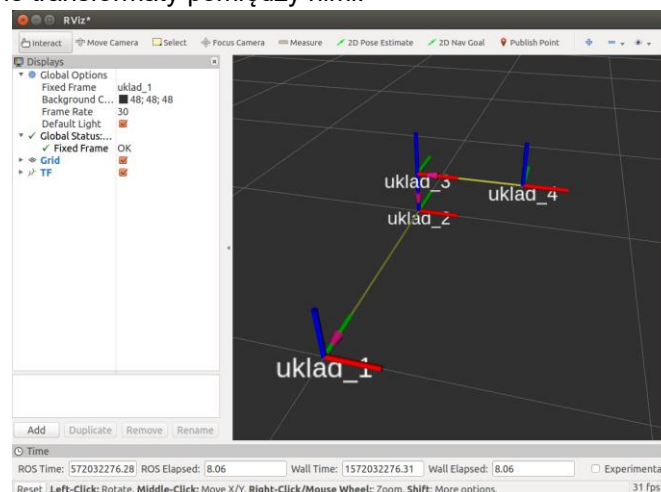
Następnie, jeśli w opcjach Displays nie jest widoczna podgrupa TF należy kliknąć na przycisk Add i z pojawiających się dostępnych opcji należy wybrać zakładkę "By display type", a następnie TF.



Z panelu *Global Options* dla pola *Fixed Frame* wybieramy **uklad_1**



Ostatecznie po uruchomieniu powinien pojawić się następujący widok układów współrzędnych i zaznaczone zdefiniowane transformaty pomiędzy nimi.



Zadanie 1

Należy uruchomić node'a turtlesim odpowiadającego za wyświetlanie żółwi (patrz poprzednie zajęcia).

- Należy napisać własny broadcaster, który będzie publikował transformatę pomiędzy układem 'world', a naszym żółwiem ('turtle1'). Do uzupełnienia transformaty danymi o **położeniu i orientacji**, proszę wykorzystać umiejętności zdobyte podczas poprzednich zajęć z subscriberem. Do przeliczenia aktualnej orientacji wyrażonej w kątach Eulera na Quaternion można zastosować następującą funkcję z biblioteki tf, która zwraca odpowiedni typ danych, którymi należy uzupełnić odpowiednie pole transformaty:

```
tf.transformations.quaternion_from_euler(roll_angle, pitch_angle, yaw_angle)
```

- Należy tak zmodyfikować plik z zadania 1a, aby publikował transformatę dla dwóch żółwi jednocześnie. Drugiego żółwia należy nazwać 'turtle2'. (UWAGA: Nawet jeśli drugi żółw nie będzie jeszcze istniał, drugi tf_broadcaster w niczym nie przeszkadza. Zacznie publikować dane o stworzonym układzie współrzędnych, w momencie, gdy kolejny żółw się pojawi.) Gdy plik zostanie zmodyfikowany można dodać drugiego żółwia w miejscu x=4, y=2, z orientacją zerową, za pomocą poznanego wcześniej serwisu.

Listener

Przykład 2

Uruchamiamy plik `tf_listener.ipynb`. Ponownie uruchomić launcha od `turtlebot3_model.launch`. W oknie Jupyter'a powinna zacząć wyświetlać się transformata od układu `base_link` do `base_scan`.

Zadanie 2

Należy napisać `tf_listener`, dzięki któremu możliwym będzie zrealizowanie "pościgu" żółwia 2 za żółwiem 1. Na podstawie aktualnego położenia żółwia 1, trzeba będzie wyliczać i zadawać prędkości żółwiowi 2 (należy do tego wykorzystać zapytanie z przykładu 2 dostarczające informacje o transformacie pomiędzy dwoma konkretnymi układami współrzędnych).

Do wyliczania prędkości liniowej i kątowej żółwia 2 można wykorzystać poniższe dwie linijki:

```
angular = 4 * math.atan2(trans[1], trans[0])
linear = 0.5 * math.sqrt(trans[0] ** 2 + trans[1] ** 2)
```

Zadania do wykonania

Zadanie 3

Uruchomić plik `roslaunch` z modelem Turtlebota3 - Burgera wymieniony we wstępie. Następnie bazując na istniejących już układach współrzędnych, dopisać własny broadcaster, który będzie publikował dodatkowy TF z układem uzależnionym od orientacji układu `wheel_right_link`. Stworzony układ o nazwie `'uklad_1'` ma być oddalony od osi koła o 0,5 m po osi Y oraz ma zachowywać cały czas tą samą orientację co układ `'base_link'`.